JNN COLLEGE OF ARTS AND SCIENCE

Team members:
Karthick B (leader)
Kamesh R
Kishore V
Mohammad Muzammil M

LearnMate - AI assistant for smarter learning and study support

Description

LearnMate is an AI-powered learning assistance platform designed to simplify the study process for students. The application allows students to enter or upload study materials, generate concise summaries, create flashcards, produce quizzes, and receive personalized study plans using the Google Gemini AI API.

The application is implemented using a lightweight web stack consisting of HTML, CSS, and JavaScript for the user interface, PHP for server-side processing and application logic, and MySQL for structured data storage. Docker is used to package the PHP application and MySQL environment so that development and deployment can be performed consistently.

The PHP application follows a modular MVC-style organization. Authentication uses PHP sessions and secure password hashing. Protected pages verify the authenticated session and user role before allowing access to student or administrator functionality. The Gemini API is accessed from the PHP backend so that the API key is not exposed in browser-side JavaScript.

2. Scenario-Based Case Study – Background

Background

Rahul is a second-year engineering student preparing for multiple semester examinations while simultaneously learning new technical skills for internships and placement opportunities. His study materials are scattered across lecture notes, PDFs, textbooks, online articles, and handwritten notes, making it difficult to organize and revise effectively.

Due to the large volume of content and limited preparation time, Rahul often struggles to identify important concepts, create revision notes, and evaluate his understanding before examinations.

Traditional study methods require significant manual effort to summarize lengthy materials, prepare practice questions, and create revision schedules. This process is repetitive, time-consuming, and can result in inconsistent learning outcomes.

Problem

- Difficulty understanding lengthy study materials within limited time.
- Manual preparation of notes consumes significant study time.
- Creating flashcards and revision questions is repetitive and labor-intensive.
- Lack of personalized study plans based on available time and examination schedules.
- No centralized platform to securely store generated learning resources.
- Difficulty assessing knowledge through self-generated quizzes.
- Switching between multiple applications for notes, quizzes, and scheduling reduces productivity.

3. Solution, Usage and Outcome

Solution

The LearnMate application provides a centralized AI-powered learning platform that automates several academic tasks. Authenticated students can enter or upload study materials and select the required AI operation. PHP processes the request, communicates with Google Gemini, and stores the generated result in MySQL.

The system performs the following operations:

- Authenticates users using PHP sessions.
- Stores users and learning resources in MySQL.
- Sends study content to the Google Gemini API through a PHP service module.
- Generates concise summaries for faster revision.
- Creates flashcards for active recall learning.
- Produces multiple-choice quizzes for self-assessment.
- Generates personalized study plans based on preferences and examination timelines.
- Stores generated learning resources for future retrieval.

Usage

Students register and log in before accessing learning features. After authentication, they can enter study material, select summary, flashcard, quiz, or study-plan generation, review the result, and save it for future reference.

Administrators manage registered users, monitor application activity, oversee AI service usage, and maintain platform integrity.

Outcome

By combining AI with a conventional PHP/MySQL web architecture, LearnMate reduces repetitive study preparation and provides students with a centralized learning workspace. Docker improves consistency between development, testing, and deployment environments.

4. Software Requirements

Software Requirements

• Operating System: Windows 10/11, macOS, or Linux with Docker support.
• Web Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or equivalent modern browser.
• HTML5: Provides the structure of the web pages.
• CSS3: Provides responsive styling and user-interface presentation.
• JavaScript: Handles client-side validation, dynamic interaction, AJAX/fetch requests, and UI updates.
• PHP 8.x: Server-side application logic and MVC implementation.
• MySQL 8.x: Relational database for users, study materials, summaries, flashcards, quizzes, and study plans.
• Apache: Web server used to serve the PHP application inside the Docker container.
• Docker and Docker Compose: Containerize the PHP/Apache application and MySQL database.
• Google Gemini API: AI service for generating educational content.
• Postman: Optional API/request testing toolkit.
• Visual Studio Code or similar IDE: Development environment.

Technology Stack

Frontend: HTML5, CSS3, JavaScript
Backend: PHP 8.x
Database: MySQL 8.x
AI: Google Gemini API
Containerization: Docker / Docker Compose
Web Server: Apache

5. Epic 1: Project Architecture

The LearnMate application follows a modular MVC-oriented architecture designed to provide secure, maintainable, and AI-powered learning services. The browser loads HTML pages styled with CSS and enhanced by JavaScript. JavaScript communicates with PHP endpoints using standard HTTP requests or fetch/AJAX calls.

PHP receives the request, validates the session and input, performs application logic, communicates with MySQL through PDO or MySQLi, and calls the Google Gemini API when AI generation is required.

The architecture separates presentation, authentication, business logic, database access, and AI service operations. This separation makes the application easier to maintain and allows individual modules to be modified without affecting the entire system.

Logical flow

Browser → HTML/CSS/JavaScript → PHP Controller → Service/Model → MySQL
■ Gemini API

Docker provides the runtime environment for the PHP/Apache application and MySQL database.

6. Components Definition

Client Application: HTML, CSS and JavaScript provide registration, login, dashboard, study-material input, AI feature selection, and resource display.

Apache/PHP Server: Apache serves PHP pages and requests. PHP controllers process form submissions, AJAX requests, sessions, validation, and application logic.

Authentication Module: PHP sessions maintain authenticated state. Passwords are stored using PHP's password_hash() function and verified with password_verify().

Controllers: PHP controller files receive requests, validate input, call services/models, and return HTML or JSON responses.

AI Service Layer: A PHP service communicates with the Google Gemini API and returns generated summaries, flashcards, quizzes, and study plans.

Database Layer: MySQL stores users, study materials, summaries, flashcards, quizzes, quiz questions, and study plans. PDO prepared statements are used for safer database interaction.

Docker Layer: Docker Compose coordinates the PHP/Apache container and MySQL container. Environment variables are used for database and Gemini configuration.

7. Entity-Relationship Diagram Description

The LearnMate relational database uses normalized MySQL tables connected through primary and foreign keys. The relational design replaces the document collections described in the original implementation.

1. User Entity
Stores registered students and administrators.
Attributes: user_id (Primary Key), name, email (Unique), password_hash, role, created_at.

2. Study Material Entity
Stores content supplied by students.
Attributes: material_id (Primary Key), user_id (Foreign Key → User), title, subject, content, file_path, created_at.

3. Summary Entity
Stores AI-generated summaries.
Attributes: summary_id (Primary Key), user_id (Foreign Key → User), material_id (Foreign Key → Study Material), summary, created_at.

4. Flashcard Entity
Stores AI-generated flashcards.
Attributes: flashcard_id (Primary Key), user_id (Foreign Key → User), material_id (Foreign Key → Study Material), question, answer, created_at.

8. Database Entities and Relationships

5. Quiz Entity
Stores generated quizzes.
Attributes: quiz_id (Primary Key), user_id (Foreign Key → User), material_id (Foreign Key → Study Material), title, created_at.

6. Quiz Question Entity
Stores individual multiple-choice questions belonging to a quiz.
Attributes: question_id (Primary Key), quiz_id (Foreign Key → Quiz), question_text, option_a, option_b, option_c, option_d, correct_option.

7. Study Plan Entity
Stores personalized study plans.
Attributes: plan_id (Primary Key), user_id (Foreign Key → User), subject, study_plan, exam_date, created_at.

Key Relationships
• User → Study Material: One-to-Many.
• Study Material → Summary: One-to-One or One-to-Many depending on revision history requirements.
• Study Material → Flashcard: One-to-Many.
• Study Material → Quiz: One-to-Many.
• Quiz → Quiz Question: One-to-Many.
• User → Study Plan: One-to-Many.
• User → AI Resources: One-to-Many.

Foreign keys maintain referential integrity and support reliable retrieval of user-specific resources.

9. Key Features and Roles

Key Features

• Secure User Authentication: PHP sessions and password_hash/password_verify provide authentication and protected access.
• Study Material Management: Students can enter, upload, view, and manage study materials.
• AI Summary Generation: Gemini generates concise summaries from supplied content.
• AI Flashcard Generation: Gemini creates question-and-answer flashcards for revision.
• AI Quiz Generation: Gemini produces multiple-choice questions for self-assessment.
• AI Study Plan Generation: Gemini creates plans according to learning goals, available time, and examination date.
• Role-Based Access Control: Student and administrator privileges are handled by session role checks.
• MySQL Data Storage: Relational tables store users and generated resources.

Student

Students can register, log in, upload or enter study material, generate AI resources, and manage their saved learning content.

Administrator

Administrators can view registered users, manage accounts, monitor usage, and maintain system-level operations.

10. User Flow

The LearnMate application begins when a user registers or logs in. PHP validates the submitted credentials and creates a session after successful authentication.

After login, the user reaches the dashboard. The student can enter study material or upload a supported file. JavaScript submits the request to a PHP endpoint. PHP validates the session and input, stores the material in MySQL, and can send the content to the Gemini API.

The user selects an AI function such as Summary, Flashcards, Quiz, or Study Plan. The PHP AI service builds the request, sends it to Gemini, validates the response, and stores the generated result in the relevant MySQL tables.

The generated resource is returned to the browser and displayed using HTML/CSS. Previously generated resources can be retrieved from MySQL through the dashboard.

Administrators follow a separate protected flow where PHP checks the user's role before showing administration pages.

11. MVC Architecture Pattern

The LearnMate application follows the Model-View-Controller pattern to organize PHP code into separate responsibilities.

Model Layer – Data Layer
PHP model classes or repository functions handle MySQL operations. They retrieve, insert, update, and delete users, study materials, summaries, flashcards, quizzes, and study plans. PDO prepared statements are used for database operations.

View Layer – Presentation Layer
HTML pages form the view layer. CSS controls presentation and responsive layout, while JavaScript adds dynamic interactions, client-side validation, asynchronous requests, and result rendering.

Controller Layer – Business Logic
PHP controllers process requests, validate input, check sessions and roles, invoke models/services, call Gemini when required, and return HTML or JSON responses.

MVC Benefits
Separating presentation, business logic, and data access improves readability, testing, debugging, and future maintenance.

12. Epic 2: Project Setup and Configuration

Creating the Project Folder

1. Create a folder named **LearnMate**.
2. Open the folder in Visual Studio Code.
3. Create the PHP application structure.
4. Add Docker configuration files.

Suggested Project Structure

LearnMate/
■■■ public/
■ ■■■ index.php
■ ■■■ login.php
■ ■■■ register.php
■ ■■■ dashboard.php
■ ■■■ assets/css/style.css
■ ■■■ assets/js/app.js
■■■ app/
■ ■■■ controllers/
■ ■■■ models/
■ ■■■ services/
■ ■■■ views/
■■■ config/
■ ■■■ database.php
■■■ uploads/
■■■ database/
■ ■■■ schema.sql
■■■ Dockerfile
■■■ docker-compose.yml
■■■ .env

This structure keeps browser-facing files, PHP logic, configuration, and database scripts separated.

13. Environment Configuration

The PHP application uses environment variables for deployment-specific configuration. Sensitive credentials should not be hard-coded into PHP or JavaScript files.

Example configuration values

APP_ENV=development
DB_HOST=mysql
DB_PORT=3306
DB_NAME=learnmate
DB_USER=learnmate_user
DB_PASSWORD=your_database_password
GEMINI_API_KEY=your_gemini_api_key

The Docker Compose service name **mysql** can be used as the database hostname from the PHP container.

Dockerfile responsibilities

- Start from a PHP-Apache image.
- Enable required PHP extensions such as PDO MySQL.
- Copy the application into the Apache document root.
- Configure Apache for the PHP application.

Docker Compose responsibilities

- Build and run the PHP/Apache service.
- Start the MySQL service.
- Create the database and persistent database volume.
- Pass environment variables to the services.

14. Epic 3: Backend Development

The LearnMate backend is developed using PHP and follows a modular MVC-oriented structure. Unlike the original Node.js implementation, the server-side application does not require npm or Express.js.

PHP handles authentication, validation, database access, file processing, AI requests, and administrative operations. HTML/CSS/JavaScript form the client-facing layer.

Backend Modules

• Authentication controller – registration, login, logout, session validation.
• Material controller – material upload, storage, retrieval, and deletion.
• AI controller – summary, flashcard, quiz, and study-plan generation.
• Admin controller – user and system management.
• Database class – reusable MySQL connection through PDO.
• Gemini service – communicates with Google Gemini API.
• Upload handler – validates file type and manages stored files.

PHP sessions protect authenticated pages. Role checks are performed before administrator functions are executed.

15. Core System Script Implementations

1. Application Entry Point
The main PHP entry point loads configuration, initializes the session, loads required classes, and routes the request to the appropriate controller or page.

2. Database Connection (config/database.php)
A reusable PDO connection establishes communication with MySQL. The connection uses environment variables and exception-based error handling.

3. Authentication Module
Registration validates the email and password, hashes the password with password_hash(), and stores the account in MySQL. Login retrieves the account and verifies the password with password_verify(). A successful login creates a PHP session.

4. AI Integration (app/services/GeminiService.php)
The Gemini service sends prompts to the Google Gemini API using PHP's HTTP client facilities such as cURL. The API key is stored server-side.

5. File Upload Module
Uploaded study material is validated for allowed file types and size before being stored. The database stores the relevant file path and metadata.

16. Epic 4: Database Configuration

LearnMate uses MySQL as its primary relational database. The database replaces the MongoDB/Mongoose implementation from the original version.

Database Configuration
The PHP application connects to MySQL through PDO. The database can run in a dedicated Docker container and uses a persistent Docker volume so that database data survives container recreation.

Normalization
The database is organized into separate related tables for users, materials, summaries, flashcards, quizzes, quiz questions, and study plans. Primary keys uniquely identify records and foreign keys maintain relationships.

Security
Prepared statements help prevent SQL injection. User passwords are stored as secure password hashes rather than plain text. Database credentials and Gemini API keys are supplied through environment configuration.

Docker MySQL
Docker Compose initializes the MySQL service with the configured database name, user, and password. A named volume provides persistent storage.

17. MySQL Schema Definitions

1. Users Table
users(user_id PK, name, email UNIQUE, password_hash, role, created_at)

2. Study Materials Table
study_materials(material_id PK, user_id FK, title, subject, content, file_path, created_at)

3. Summaries Table
summaries(summary_id PK, user_id FK, material_id FK, summary, created_at)

4. Flashcards Table
flashcards(flashcard_id PK, user_id FK, material_id FK, question, answer, created_at)

5. Quizzes Table
quizzes(quiz_id PK, user_id FK, material_id FK, title, created_at)

6. Quiz Questions Table
quiz_questions(question_id PK, quiz_id FK, question_text, option_a, option_b, option_c, option_d, correct_option)

7. Study Plans Table
study_plans(plan_id PK, user_id FK, subject, study_plan, exam_date, created_at)

Indexes should be added to frequently queried foreign-key columns such as user_id and material_id.

18. Database Tables and API Interface

Database Tables

• Users – stores account and authentication information.
• Study Materials – stores user-provided learning content.
• Summaries – stores AI-generated summaries.
• Flashcards – stores AI-generated question/answer pairs.
• Quizzes – stores generated quizzes.
• Quiz Questions – stores individual quiz questions.
• Study Plans – stores personalized schedules.

Application Interfaces

Although the application can operate using normal PHP pages and forms, JavaScript can communicate with PHP endpoints using fetch/AJAX for a smoother interface.

Authentication

POST /api/auth/register.php
POST /api/auth/login.php
POST /api/auth/logout.php

Study Material

POST /api/material/upload.php
GET /api/material/list.php
GET /api/material/view.php?id=...

AI Features

POST /api/ai/summary.php
POST /api/ai/flashcards.php
POST /api/ai/quiz.php
POST /api/ai/study-plan.php

Administrative

GET /api/admin/users.php

19. Epic 5: Project Execution

The LearnMate application can be executed using Docker Compose. Docker provides a repeatable environment containing the PHP/Apache application and MySQL database.

Execution Steps

1. Install Docker Desktop or Docker Engine with Compose support.
2. Open the LearnMate project directory in a terminal.
3. Configure the .env file with MySQL and Gemini settings.
4. Build the containers using Docker Compose.
5. Start the services.
6. Open the application through the mapped Apache port.

Typical commands

```
docker compose build
docker compose up -d
docker compose ps
docker compose logs
```

The first startup initializes the MySQL database and creates the required tables using the supplied schema.sql or initialization process.

Application Testing

Test registration, login, dashboard access, material upload, AI generation, resource storage, logout, and administrator access. Browser developer tools can be used to inspect JavaScript/fetch requests.

20. API Testing – Authentication

1. User Registration
Registers a new user and stores the securely hashed password in MySQL.

Endpoint:
POST /api/auth/register.php

Example form fields:
name = Rahul
email = rahul@example.com
password = ********

PHP validates the fields, checks whether the email already exists, hashes the password using password_hash(), inserts the user record, and returns a success response.

2. User Login
Authenticates the registered user and creates a PHP session.

Endpoint:
POST /api/auth/login.php

The login process retrieves the user by email, verifies the submitted password using password_verify(), stores user_id, name, and role in the session, and redirects or returns a JSON success response.

Security
Protected pages check the session before serving private resources. Administrator endpoints additionally verify the session role.

21. API Testing – Study Material and Summary

3. Upload Study Material
Allows an authenticated student to provide study material for AI processing.

Endpoint:
POST /api/material/upload.php

The PHP upload handler validates the request, checks the session, stores the file or text content, and inserts a study_materials record into MySQL.

4. Generate AI Summary
Sends the selected material to the Gemini API and stores the generated concise summary.

Endpoint:
POST /api/ai/summary.php

The PHP controller verifies that the material belongs to the logged-in user, retrieves its content from MySQL, calls Gemini through GeminiService.php, stores the generated summary, and returns the result.

JavaScript can display a loading state while the request is processed and then render the generated summary in the dashboard.

The same security pattern should be applied to every AI endpoint so that users cannot request or modify another user's learning resources.

22. API Testing – Flashcards, Quiz and Study Plan

5. Generate AI Flashcards
Creates AI-generated flashcards from the selected study material.
Endpoint: POST /api/ai/flashcards.php

6. Generate AI Quiz
Generates multiple-choice questions and stores them in the quizzes and quiz_questions tables.
Endpoint: POST /api/ai/quiz.php

7. Generate AI Study Plan
Creates a personalized study schedule based on the user's learning goals, available study time, subject, and examination date.
Endpoint: POST /api/ai/study-plan.php

Processing Flow
Browser → JavaScript fetch() → PHP Controller → Authentication Check → MySQL → Gemini API → MySQL → JSON Response → HTML Dashboard

The application can also expose conventional PHP pages instead of JSON endpoints. The endpoint structure is provided to keep the implementation modular and make future mobile or frontend integration easier.

Thank You

LearnMate combines HTML, CSS, JavaScript, PHP, MySQL, Google Gemini AI, and Docker into a practical AI-assisted learning platform.